Automata

languages

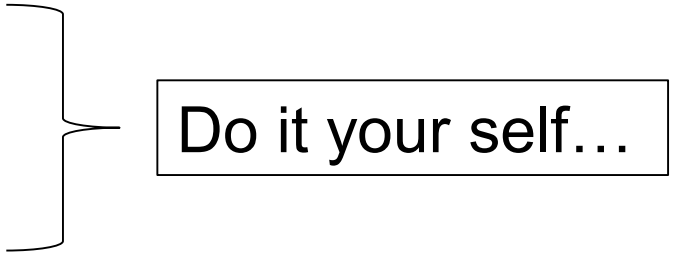
lexers, parsers, interpreters

5

- Teacher: Hans van Heumen

Week1 schedule

1. Grammar
2. ANTLR
3. assignments



Introduction Grammars

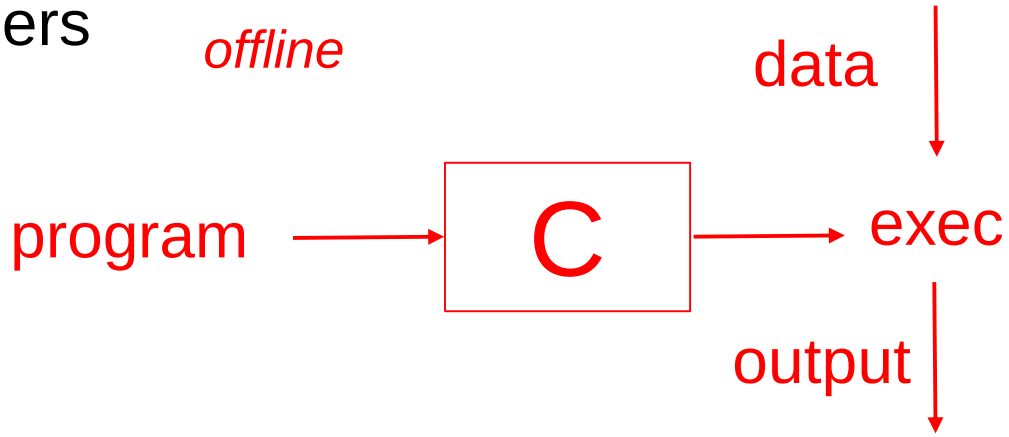
Some terms

- Symbols
  - Smallest unit
  - (0,1,a,b,…)
- Alphabet
  - Finite Set of symbols
  - ∑(a,b)
- Strings
  - Sequence of alphabet
- Language
  - Collection of strings

Intro Compilers

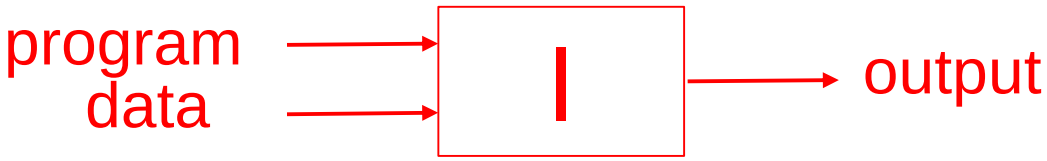
Compilers

offline



Interpreters

online



Intro Compilers

1. Lexical Analysis
2. Parsing

Syntactic aspects

3. Semantic Analysis — *Semantic aspects such as types and scope*
4. Optimization — *Collection of transformations on the program to make it run faster or memory optimization*
5. Code generation. — *Translation to another language*

Lexical analysis

Lexical analysis

1. Divides program text into “words” also called “lexemes”

This is a sentence.

if x == y then z = 1; else z = 2;

Lexical analysis

Lexical analysis

1. Divides program text into “words” also called “lexemes”
2. Classifies lexemes in roles called “token classes”
   Example of token classes:

This	line	is	a	longer	sentence
Article	Noun	Verb	Article	Adjective	Noun

Lexical analysis

Lexical analysis

1. Divides program text into “words” also called “lexemes”
2. Classifies lexemes in roles called “token classes”

Example of token classes:

```
if (i == j)
    Z = 0;
else
    Z = 1;
```

Operator
Whitespace
Identifier
Keyword
Number

( ) ; =

Token classes with single string element

\tif (i == j)\n\t\tz = 0;\n\telse\n\t\tz = 1;

w k w (l w o w i) w w w = n ; w w k w w w i w = w n ;

Lexical specification

Lexical specification of programming languages

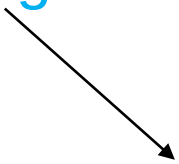
Using regular expressions to describe different constructs of a programming language

Example:

Integers: a non-empty string of digits

digit = ‘0’ + ’1’ + ’2’ + ’3’ + ’4’ + ’5’ + ’6’ + ’7’ + ’8’ + ’9’

digits = digit digit*



the regular expression for Integer numbers (token class)

Lexical specification

Lexical specification of programming languages

Using regular expressions to describe different constructs of a programming language

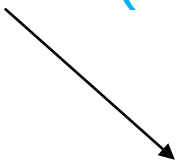
Example:

Identifiers: string of letters and digits, starting with a letter

letter = ‘a’ + ’b’ + …+ ’Z’

digit = ‘0’ + ’1’ + ’2’ + ’3’ + ’4’ + ’5’ + ’6’ + ’7’ + ’8’ + ’9’

identifiers = letter (letter + digit)*

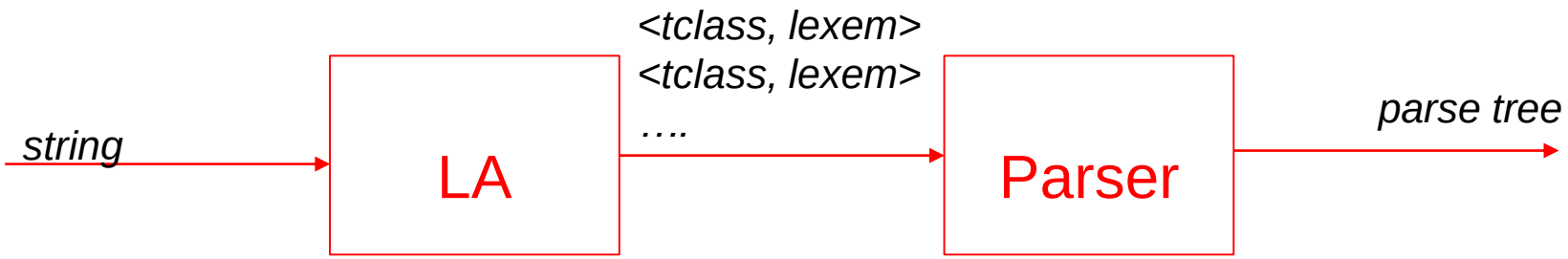


the regular expression for Identifiers (token class)

Lexical analysis

Lexical analysis

1. Divides program text into “words” also called “lexemes”
2. Classifies lexemes in roles called “token classes”
3. Communicate tokens = <token class, lexem> to the parser



Lexical analysis: Parsing

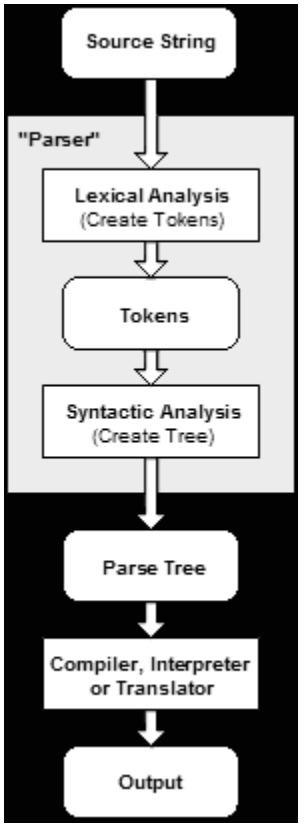
Parsing

- Once words are understood, the next step is to understand *sentence structure*
  - *Word (token) types*
  - *Grouping them in more complex structures*
  - *Up to the whole sentence*

- Diagramming Sentences
  - The diagram is a tree

Parser

A parser is commonly used as a component of an interpreter or a compiler. The overall process of parsing involves three stages:



1.Lexical Analysis: A lexical analyzer is used to produce tokens from a stream of input string characters, which are broken into small components to form meaningful expressions.

- "12 * (3 + 4)^2" and split it into the tokens 12, *, (, 3, +, 4,), ^, 2,

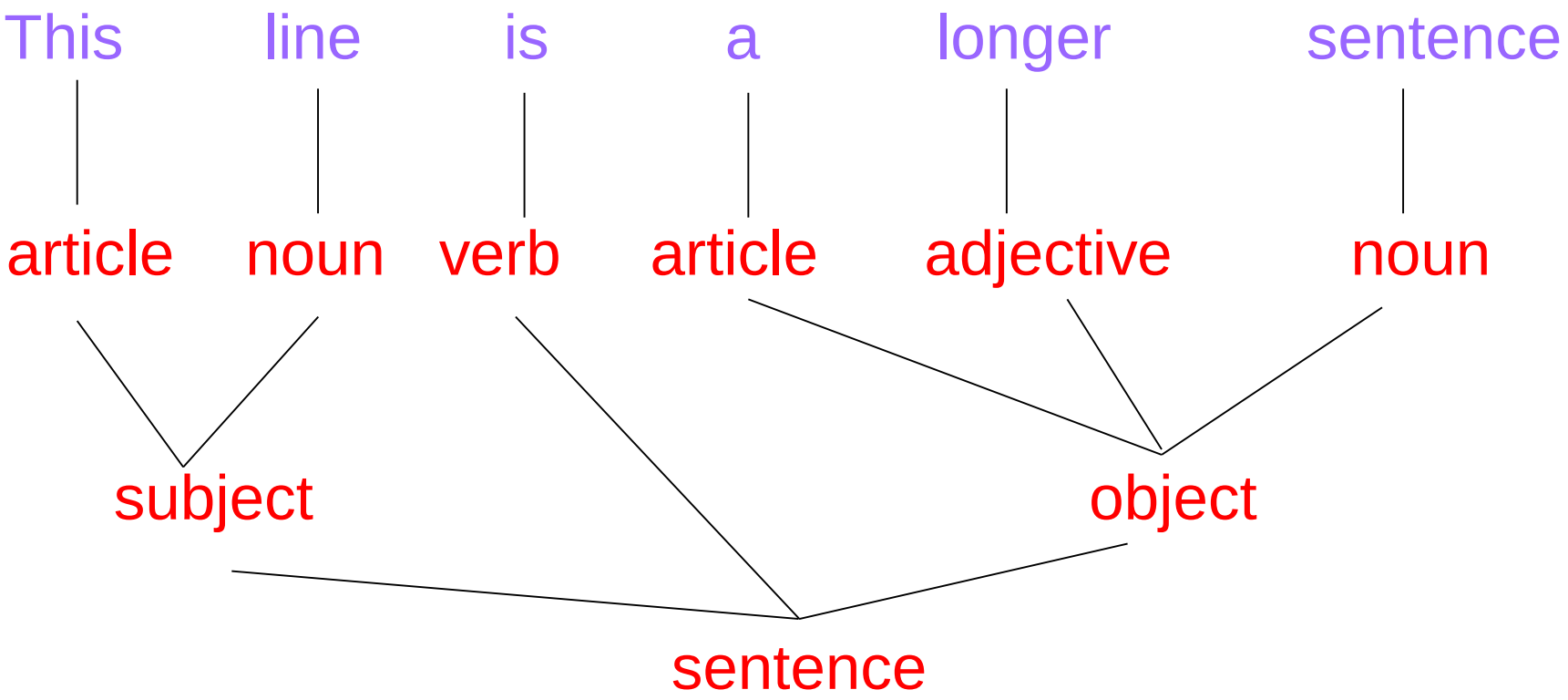
2.Syntactic Analysis: Checks whether the generated tokens form a meaningful expression. This makes use of a context-free grammar that defines algorithmic procedures for components. These work to form an expression and define the particular order in which tokens must be placed.

- "12 *+* (3 + 4)^2“ is not meaningful

3.**Semantic Parsing**: The final parsing stage in which the meaning and implications of the validated expression are determined and necessary actions are taken.

- A compiler would generate executable code

Lexical analysis: Parsing



Lexical analysis: Parsing

if x == y then z = 1; else z = 2;

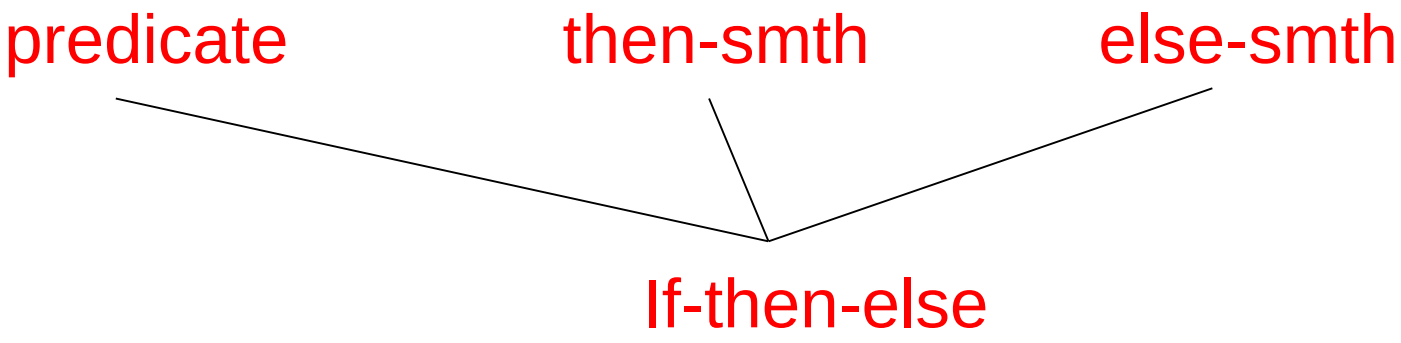
x == y z 1 z 2

If-then-else

Lexical analysis: Parsing

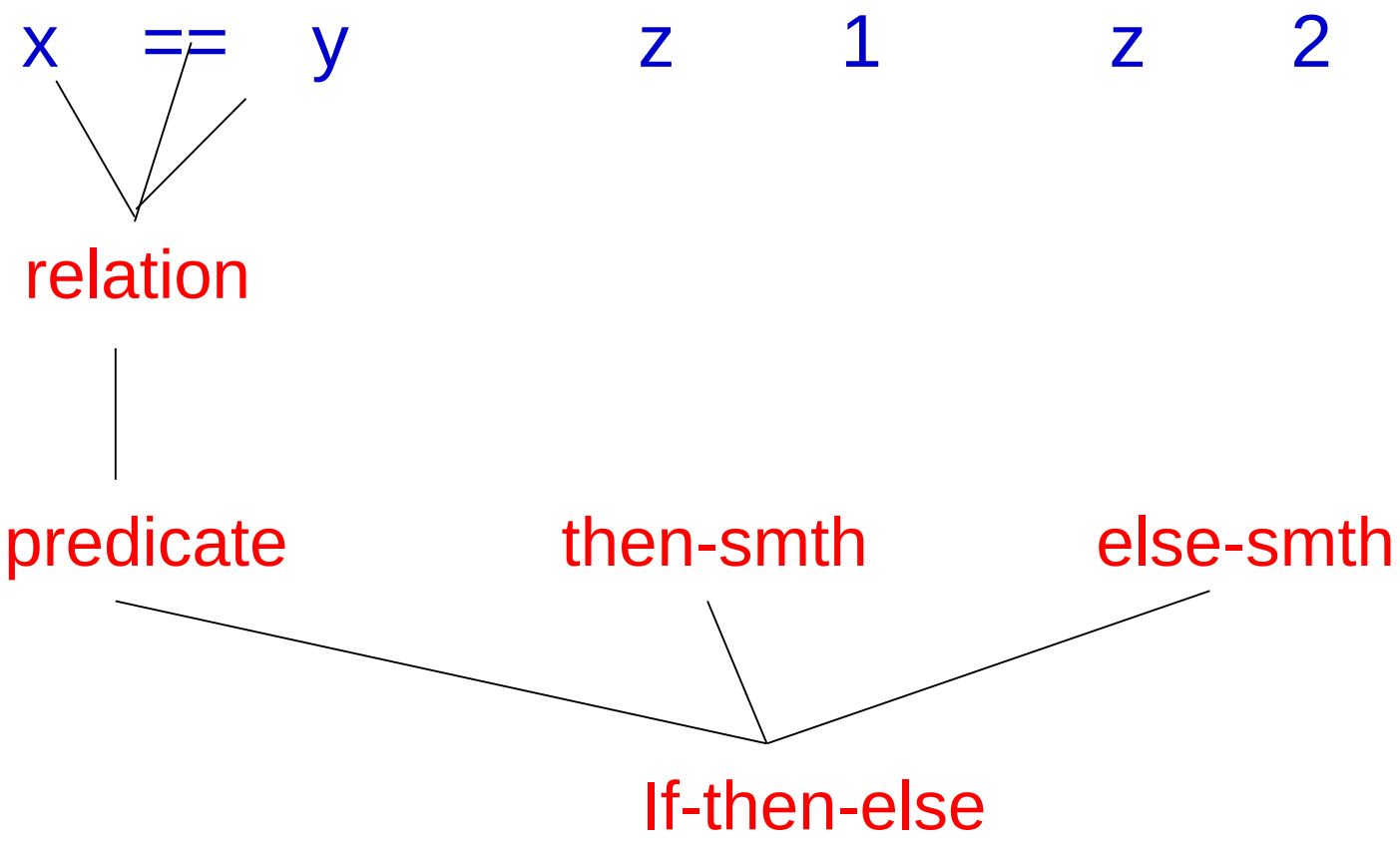
if x == y then z = 1; else z = 2;

x == y z 1 z 2



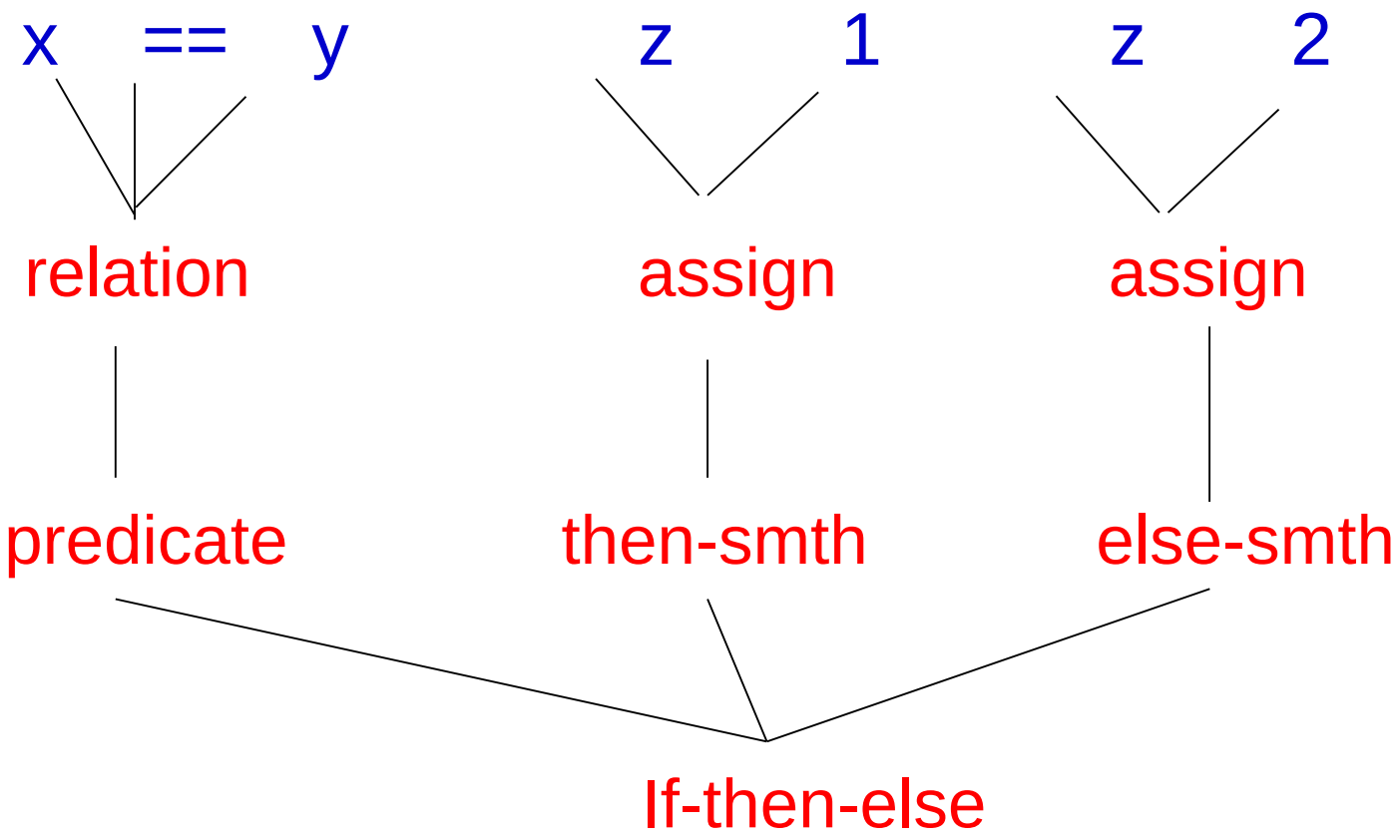
Lexical analysis: Parsing

if x == y then z = 1; else z = 2;



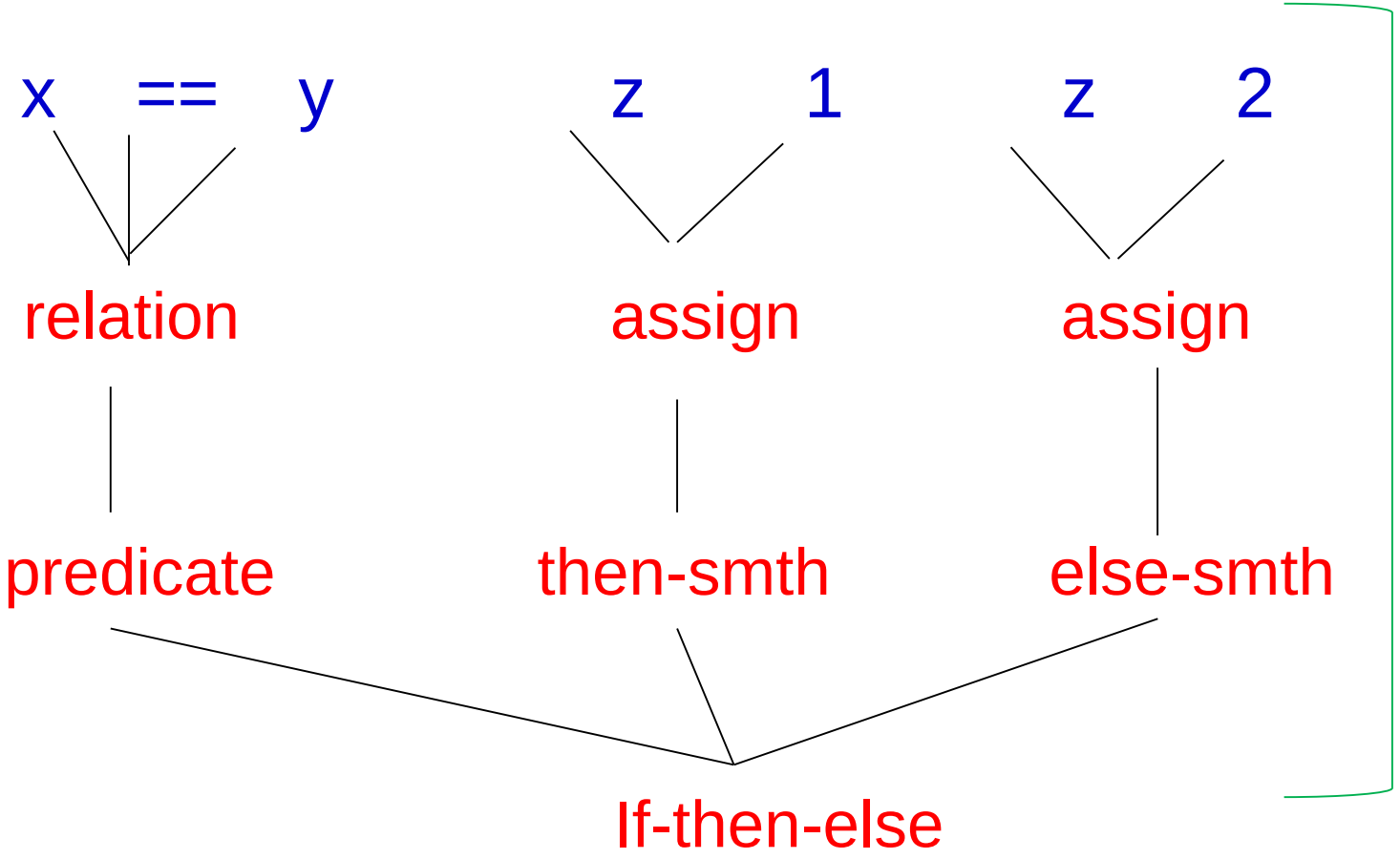
Lexical analysis: Parsing

if x == y then z = 1; else z = 2;



Lexical analysis: Parsing

if x == y then z = 1; else z = 2;



Parse tree showing structure braking in its constituent pieces

Abstract Syntax Tree (AST)

- An abstract syntax tree represents all of the syntactical elements of a programming language, similar to syntax trees that linguists use for human languages.
- The tree focuses on the rules rather than elements like braces or semicolons that terminate statements in some languages.
- The tree is hierarchical, with the elements of programming statements broken down into their parts.
- For example, a tree for a conditional statement has the rules for variables hanging down from the required operator.
- ASTs are widely used in compilers to check code for accuracy.
- If the generated tree contains errors, the compiler prints an error message.
- ASTs are used because some constructs cannot be represented in a context-free grammar, such as implicit typing.
- ASTs are highly specific to programming languages, but research is underway on universal syntax trees.

Parsing

A parser's main purpose is to determine if input data may be derived from the start symbol of the grammar. If yes, then in what ways can this input data be derived? This is achieved as follows:

•**Top-Down Parsing**: Involves searching a parse tree to find the left most derivations of an input stream by using a top-down expansion. Examples include LL parsers and recursive-descent parsers.

•**Bottom-Up Parsing**: Involves rewriting the input back to the start symbol. This type of parsing is also known as shift-reduce parsing. One example is a LR parser.

Parsers are widely used in the following technologies:

•Java and other programming languages
•HTML and XML
•Interactive data language and object definition language
•Database languages, such as SQL
•Modeling languages, such as virtual reality modeling language
•Scripting languages
•Protocols, such as HTTP and Internet remote function calls

Review

- The lexer which takes input as a string and converts the input into a set of tokens.
- The Parser which takes the tokens from the lexer and returns a syntax tree based on a grammar. The grammar is often expressed in a meta language.
- The grammar is the language of languages and provides the rules and syntax.
- The interpreter which takes in the syntax tree and evaluates the result. In this case we return the result of calculator input.